Per-language training loss at no measurable cost: instrumenting a chunked cross-entropy CPT trainer without changing the run

PinoAI engineering note · ENG-006 · technical report · Taglish edition: ENG-006-per-language-loss.tl.qmd

Stan Lee V. Natividad
Kooya PH AI Research
stan@kooya.ph

2026-10-06

Abstract

To teach PinoAI Filipino, a continued-pretraining (CPT) run mixes several languages and several corpus buckets in one stream. A single loss curve cannot say whether Tagalog, Ilokano or Waray is improving, or whether one bucket is being memorised. DEC-005 Amendment 16 adds a per-mix-bucket and per-document-language breakdown of the training loss, logged every step. The worry with any such instrument is that it changes the run, or that it slows the GPU. This note shows it does neither, to the precision the controls actually measured.

1. **The run is unchanged in expectation.** The amended trainer consumes the data in a bit-identical order: the same sha256 over 2,000 sequences, and over a 500-sequence resume, including when the amended code resumes a pre-A16 stream state. The training loss itself is re-expressed — a `reduction="none"` cross-entropy, summed, in place of the registered `reduction="sum"`, so the per-token values exist for the breakdown — and a control holds it to the pre-A16 loss within relative 1e-6 with gradients `allclose`. So the run is unchanged in expectation, to floating-point summation order, not bit for bit on the loss.
2. **No measurable GPU cost, and a sub-0.1% CPU cost.** On one H100, the LM head plus cross-entropy takes 329.05 ms without the metric and 329.16 ms with it; the 0.11 ms difference sits inside overlapping inter-quartile ranges across 40 interleaved samples each. The CPU-side bookkeeping (building the per-token index and scattering into per-bucket and per-language sums) is measured separately at about 2.5 ms, which is 0.086% of a 2.876 s micro-batch.
3. **The one big number is an artefact.** A CPU micro-benchmark on a tiny model shows the metric running at 0.864 of the un-instrumented rate. That is a tiny model with no GPU and no real head, where the fixed bookkeeping is a large share of a small total. It is reported as an upper bound, and it is exactly why a tiny benchmark must not be used to judge a GPU-path cost.
4. **Why it matters.** A per-language metric is only as honest as the per-language data behind it. The r3b corpus audit found a Waray source that was 48 of 50 one bot-generated sentence frame; without the audit, this metric would have shown that source as fast "Waray learning". The instrument is necessary; it is not sufficient.

1. What the metric is for, and what could go wrong

PinoAI's first CPT run trains Gemma 4 E4B on a mixture: Tagalog, several regional languages, an English replay, a literary slice and a lexicon. The headline is one cross-entropy loss. That number cannot answer the questions the run is actually asking. Is Ilokano improving, or is the average being carried by Tagalog? Is a small bucket being memorised (loss falling to near zero) while a large one barely moves? Those are per-bucket and per-language questions, and the registered per-language held-out evaluation (C1) answers them only at the eval points, not step by step.

Amendment 16 adds the step-by-step view: for every optimizer step, the training loss broken down by mix bucket and by the language of the document each token came from. The design risk is the usual one for instrumentation. A metric that reads the model's state can perturb it; a metric that adds work to the inner loop can cost GPU time, which on a ~117-hour run is real money. Before trusting the curves, the amendment had to show it does neither. The evidence is the controls below.

2. The design

The breakdown is computed from the **same** logits the training loss already uses, not from a second forward pass.

- The trainer computes cross-entropy in chunks over the vocabulary to bound memory (the 262,144-token Gemma vocabulary makes a single full-batch softmax expensive). To produce per-token values, Amendment 16 computes each chunk's cross-entropy with `reduction="none"` and then sums it, where the pre-A16 trainer used `reduction="sum"` directly. The two are equal **to floating-point summation order**; there is no second matmul.
- The per-token negative log-likelihoods used for the breakdown are **detached**, so the breakdown adds no gradient path. The gradient still flows through the summed loss, exactly as before.
- Each per-token value is attributed to the **target** token's bucket and language, read from the data stream's labels, not guessed. A document whose language is unknown is tagged `untagged`; it is never assigned a language by heuristic.
- There is no synthetic beginning-of-sequence target, so the token counts match the real supervised tokens.
- The per-step record carries token-weighted means per bucket and per language; a reader recovers the step's overall loss as the token-weighted mean across buckets.

The claim to check is therefore in two parts: the forward and backward computation that defines the run must be **unchanged in expectation** (data order identical; loss and gradients equal to summation order), and the extra work — on the GPU loss path and on the CPU — must not show up as a cost.

3. Four controls

3.1 The run is unchanged in expectation

Two checks, not one. The first compares the **data order**: the amended trainer and the pre-A16 (A14) trainer were run over the same data and their token streams hashed.

Table 1: Token-stream (data-order) identity, A14 vs A16 (`control_stream_identity.py`).

Comparison	A14 sha256	A16 sha256	Identical
2,000 sequences × 64 tokens	731d461b	731d461b	yes
resume over 500 sequences	2eb3f805	2eb3f805	yes
resume over 500 sequences, A16 code resuming an **A14-format** stream state (no `buffer_lang`)	—	2eb3f805	yes

All three match: the amended code consumes the data in the same order, and a stream state written before A16 existed resumes under A16 to the same order. The stored **stream** state is equal except for the new per-language bookkeeping field (`buffer_lang`), which carries no gradient and feeds no update.

The second check is the **loss path**. Because A16 re-expresses the chunk cross-entropy (`reduction="none"` then sum), the loss is not bit-identical to the `reduction="sum"` original; it is equal to floating-point summation order. ml-engineer's control (`test_per_token_output_matches_the_summed_loss_and_its_gradients`) asserts the A16 loss matches the summed loss within relative 1e-6 and the gradients are `allclose`. So the metric does not change the run in expectation; it changes the summation order of one reduction, within numerical noise, and it adds no gradient.

3.2 No measurable cost on the GPU loss path

The GPU work the metric touches is the LM head and the chunked cross-entropy, where the per-token output is produced. We benchmarked that path alone at the real shape (hidden 2,560, vocab 262,144, softcap 30) on one H100, 40 timed samples per arm, interleaved to spread any thermal drift across both.

Table 2: H100 LM head + cross-entropy, per call (`control_h100_loss_bench.py`; losses.py sha256 64780a6b, the registered A16 file).

Arm	Median	Inter-quartile range
Metric off	329.05 ms	328.83 – 329.40 ms
Metric on	329.16 ms	328.90 – 329.43 ms

The difference is 0.11 ms on a 329 ms operation, about 0.03%, and the two inter-quartile ranges overlap. There is no measurable cost on the GPU loss path. The benchmark recorded the sha256 of the `losses.py` it ran (64780a6b), which is the file registered under A16, so the measured code is the shipped code.

3.3 The CPU bookkeeping, measured

The metric also does CPU-side work the H100 benchmark does **not** time: `LossBreakdown.add` (scattering per-token values into per-bucket and per-language sums) and the stream's per-sequence language tagging. These were measured directly at the E4B micro-batch shape (4 × 4,096, with 6 buckets and 11 languages).

Table 3: CPU bookkeeping (`eng006-inputs/control_cpu_bookkeeping.py`).

CPU item	Measured
`LossBreakdown.add`, per micro-batch (median of 20)	2.435 ms
stream language tagging, extra per micro-batch (A14 data.py 0d3d6a13 vs A16 60ae100b, 6 interleaved reps each)	0.0329 ms
micro-batch wall time (from the plumbing's 5,697.4 tokens/s)	2.876 s
total CPU bookkeeping as a fraction of a micro-batch	0.086%

About 2.5 ms of CPU bookkeeping sits on a 2.876 s micro-batch: 0.086%. Combined with §3.2, the full per-step cost of the metric is a GPU-path change of +0.11 ms on a 329 ms head (inside overlapping IQRs) plus ~2.5 ms of CPU work per 2.9 s micro-batch — under a tenth of a percent, on the real shape.

3.4 Why the tiny-model CPU ratio misleads

A CPU-only micro-benchmark on a tiny model tells a different-looking story, and it is worth reporting precisely because it is the trap.

Table 4: CPU throughput, tiny model, 66 steps per arm (`control_cpu_toks.py`).

Arm	Median tokens/s (CPU, tiny model)
Metric off	63,819
Metric on	55,151

That is a ratio of 0.864: the instrumented path runs at 86% of the plain one. Taken at face value it would be a 14% tax. It is not. On a CPU with a tiny model there is no large head matmul and no real softmax, so the fixed ~2.5 ms of bookkeeping (§3.3) is a large fraction of a very small per-step total. On the H100 at the real shape, that same 2.5 ms is 0.086% of the micro-batch, and the GPU loss path is unchanged (§3.2). The 0.864 is therefore an **upper bound** on the overhead, realised only in a regime the real run never enters. It is the concrete reason a tiny-model or CPU benchmark cannot be used to accept or reject a GPU-path cost.

4. Why an honest per-language metric is necessary but not sufficient

The point of the breakdown is to see which language is learning. That makes it only as trustworthy as the per-language data feeding it. The r3b corpus work made the risk concrete. An unaudited Waray Wikipedia source was, on a 50-document audit, 48 of 50 one bot-generated sentence frame (the Lsjbot species/genus stubs), about 64% of the round's Waray tokens. Trained at one epoch, a per-language curve would have shown Waray's loss dropping quickly and read as fast "Waray learning" — when the model was only memorising one template. The source was dropped and the language rebuilt from its real web and crawl text; the per-language readout for Waray is withheld until its remaining sources pass the same prose audit.

So the instrument and the data gate work together. Amendment 16 makes the per-language signal **visible** at no cost to the run; the corpus audit makes it **meaningful**. Either alone would mislead: a clean metric over template data, or clean data with only an average to look at.

5. Limitations

- **The GPU benchmark is the head and loss path in isolation.** It is the right unit for the GPU cost, but it is not an end-to-end step time. The end-to-end claim rests on §3.1 (data order identical, loss and gradients equal to summation order) plus the separately measured CPU bookkeeping of §3.3.
- **The CPU figures are model- and shape-dependent.** 2.5 ms is one E4B micro-batch shape on one machine; 0.864 is one tiny model, used only as an upper bound, not a rate.
- **The loss match is numerical, not exact.** Re-expressing the reduction changes the summation order, so the loss is equal to relative 1e-6, not bit-identical. The data order is bit-identical.
- **The metric records; it does not interpret.** A falling per-language loss can be learning or memorisation (§4). Reading it requires the data audit, and for small languages the C1 per-language readout's precision rules.

- **One trainer, one shape.** The numbers are for Gemma 4 E4B's head (hidden 2,560, vocab 262,144). A different vocab or hidden size would change the head cost, though not the data-order argument.

Reproducibility

Table 5: Files.

Item	Where
Control outputs (GPU, CPU ratio, stream)	`experiments/results/cpt/trainer-breakdown-20261006/` (`control_h100_loss_bench.out`, `control_cpu_toks.out`, `control_stream_identity.out`), on main via #76
CPU bookkeeping	`experiments/results/cpt/eng006-inputs/control_cpu_bookkeeping.{py,out}` (ml-engineer f2a06f; registered with this paper, sha in DECISION_LOG)
Numbers in this note	`experiments/reports/_eng006.py` prints and asserts each one from those outputs
Loss / gradient match	ml-engineer's `test_per_token_output_matches_the_summed_loss_and_its_gradients` (relative 1e-6; gradients allclose)
Code	`losses.py` sha256 64780a6b and `data.py` sha256 60ae100b (DEC-005 Amendment 16 real-path lines); A16 trainer sha256 c38b6c4e, superseded by A17 87b4700f for the pilot
The run-level trainer design	ENG-005
Defects and fixes	PM-017

Acknowledgments

We thank the leadership of Kooya PH for supporting this work, and the native-speaker evaluators who reviewed and rated the Filipino and Taglish material.

AI assistance. Analysis code, statistical checks and drafts of this document were prepared with the help of Claude (Anthropic), an AI model. Every number is generated from the project's result files. The author reviewed the content and takes responsibility for it.